

Лабораторна робота №2

Мета роботи

Набуття практичних навичок перетворення концептуальної моделі «сутність-зв'язок» (ERD) у реляційну схему бази даних, написання інструкцій DDL для створення таблиць, визначення первинних і зовнішніх ключів, налаштування обмежень цілісності (NOT NULL, UNIQUE, CHECK, DEFAULT) у СКБД PostgreSQL та заповнення таблиць тестовими даними.

1. Короткий виклад вимог та призначення системи

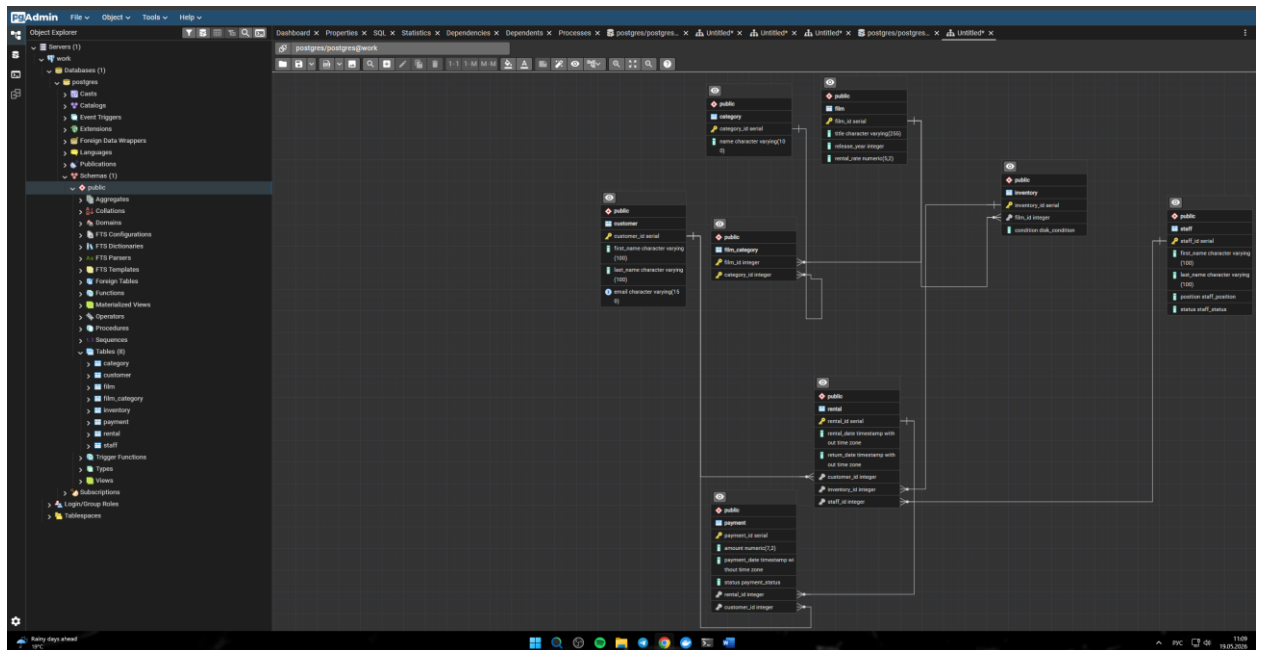
Призначення системи: Автоматизація бізнес-процесів функціонування пункту прокату фільмів. Система розробляється для ведення обліку наявного каталогу кінострічок, їх розподілу за жанрами, контролю за конкретними фізичними копіями носіїв (інвентарем) на складі, збереження даних про клієнтів та персонал, а також для фіксації кожної операції видачі/повернення фільмів та супутніх фінансових розрахунків.

Основні бізнес-правила та обмеження предметної області:

1. Один фільм може належати до кількох жанрів одночасно, а один жанр містить багато фільмів (реалізація зв'язку «багато до багатьох» через асоціативну таблицю).
 2. Кожна кінострічка з каталогу може мати декілька фізичних копій (дисків), стан яких відстежується індивідуально.
 3. Кожна транзакція оренди фіксує конкретного клієнта, одиницю інвентарю та співробітника, який оформив операцію.
 4. Будь-який факт видачі диска супроводжується фінансовим чеком із фіксацією статусу платежу та суми.
-

2. Діаграма реляційної схеми бази даних

Фізичну реляційну схему було автоматично згенеровано всередині СКБД PostgreSQL за допомогою інструменту pgAdmin ERD Tool на основі створених таблиць. Схема відображає точні типи даних стовпців, наявність індексів, первинних (PK) та зовнішніх (FK) ключів.



Пояснення до діаграми: Зв'язки між таблицями представлені в нотації «Crow's Foot» (Лпка ворони). Наприклад, зв'язок між таблицями CUSTOMER та RENTAL використовує графічний символ «нуль або багато» (кружечок та розгалуження) з боку таблиці оренди. Це відображає бізнес-логіку системи: у базі може існувати новий клієнт, який ще не здійснив жодної операції прокату.

3. Опис остаточної схеми, таблиць та обмежень цілісності

Для забезпечення максимальної несуперечливості даних та суворої валідації на рівні бази даних, перед описом таблиць було розроблено **4 кастомні типи даних (перерахування ENUM)**:

- `disk_condition` — поточний стан фізичного носія ('New', 'Good', 'Damaged', 'Lost').
- `staff_position` — посада співробітника ('Cashier', 'Manager', 'Administrator').
- `staff_status` — трудовий статус ('Active', 'On_Leave', 'Temporary').
- `payment_status` — статус фінансової операції ('Completed', 'Delayed', 'Cancelled').

Детальний опис структури таблиць:

1. CATEGORY (Жанри фільмів)

- `category_id` (PRIMARY KEY, SERIAL): Унікальний ідентифікатор категорії з автоінкрементом.
- `name` (VARCHAR(100), NOT NULL): Назва жанру.

2. FILM (Каталог кінострічок)

- `film_id` (PRIMARY KEY, SERIAL): Унікальний код фільму.
- `title` (VARCHAR(255), NOT NULL): Назва фільму.
- `release_year` (INT): Рік випуску фільму. *Обмеження:* CHECK (`release_year >= 1895 AND release_year <= 2100`) захищає від некоректного введення дат (кінематограф існує з 1895 року).
- `rental_rate` (DECIMAL(5,2), NOT NULL): Вартість оренди.

3. FILM_CATEGORY (Асоціативна таблиця зв'язку фільмів та жанрів)

- film_id (FOREIGN KEY REFERENCES FILM(film_id) ON DELETE CASCADE)
- category_id (FOREIGN KEY REFERENCES CATEGORY(category_id) ON DELETE CASCADE)
- *Особливість*: Первинним ключем є **компози́тний (складений) ключ** PRIMARY KEY (film_id, category_id). Це виключає ризик дублювання зв'язків між одним фільмом та одним жанром.

4. INVENTORY (Фізичні копії дисків)

- inventory_id (PRIMARY KEY, SERIAL): Штрихкод конкретного диска.
- film_id (FOREIGN KEY REFERENCES FILM(film_id) ON DELETE RESTRICT).
- condition (disk_condition, NOT NULL, DEFAULT 'New'): Поле використовує створений ENUM для контролю стану носія.

5. CUSTOMER (Журнал клієнтів)

- customer_id (PRIMARY KEY, SERIAL): Номер картки клієнта.
- first_name, last_name (VARCHAR(100), NOT NULL): Персональні дані.
- email (VARCHAR(150), UNIQUE): Електронна адреса. *Обмеження*: CHECK (email LIKE '%@%') виконує обов'язкову перевірку наявності символу @.

6. STAFF (Персонал пункту прокату)

- staff_id (PRIMARY KEY, SERIAL): Код працівника.
- position (staff_position, NOT NULL): Посада працівника (ENUM).
- status (staff_status, NOT NULL, DEFAULT 'Active'): Статус працівника (ENUM).

7. RENTAL (Журнал операцій оренди)

- rental_id (PRIMARY KEY, SERIAL): Номер операції.
- rental_date (TIMESTAMP, NOT NULL, DEFAULT CURRENT_TIMESTAMP): Точний час видачі.
- return_date (TIMESTAMP, NULL): Точний час повернення. Якщо дорівнює NULL — диск на руках у клієнта.
- customer_id, inventory_id, staff_id: Зовнішні ключі (FOREIGN KEY), що зв'язують операцію з клієнтом, носієм та касиром.

8. PAYMENT (Фінансовий облік)

- payment_id (PRIMARY KEY, SERIAL): Номер чека.
- amount (DECIMAL(7,2), NOT NULL): Сума до сплати. *Обмеження*: CHECK (amount > 0) блокує від'ємні або нульові транзакції.
- status (payment_status, NOT NULL, DEFAULT 'Completed'): Фінансовий статус (ENUM).
- rental_id (FOREIGN KEY REFERENCES RENTAL(rental_id) ON DELETE CASCADE), customer_id (FOREIGN KEY REFERENCES CUSTOMER(customer_id)).

4. SQL-скрипт створення структури баз даних та заповнення таблиць

```
CREATE TYPE disk_condition AS ENUM ('New', 'Good', 'Damaged', 'Lost');  
CREATE TYPE staff_position AS ENUM ('Cashier', 'Manager', 'Administrator');  
CREATE TYPE staff_status AS ENUM ('Active', 'On_Leave', 'Temporary');  
CREATE TYPE payment_status AS ENUM ('Completed', 'Delayed', 'Cancelled');
```

```
CREATE TABLE CATEGORY (  
    category_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE FILM (  
    film_id SERIAL PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    release_year INT CHECK (release_year >= 1895 AND release_year <= 2100),  
    rental_rate DECIMAL(5, 2) NOT NULL  
);
```

```
CREATE TABLE FILM_CATEGORY (  
    film_id INT REFERENCES FILM(film_id) ON DELETE CASCADE,  
    category_id INT REFERENCES CATEGORY(category_id) ON DELETE CASCADE,  
    PRIMARY KEY (film_id, category_id)  
);
```

```
CREATE TABLE INVENTORY (  
    inventory_id SERIAL PRIMARY KEY,  
    film_id INT REFERENCES FILM(film_id) ON DELETE RESTRICT,  
    condition disk_condition NOT NULL DEFAULT 'New'  
);
```

```
CREATE TABLE CUSTOMER (  
    customer_id SERIAL PRIMARY KEY,
```

```
first_name VARCHAR(100) NOT NULL,  
last_name VARCHAR(100) NOT NULL,  
email VARCHAR(150) UNIQUE CHECK (email LIKE '%@%')  
);
```

```
CREATE TABLE STAFF (  
    staff_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    position staff_position NOT NULL,  
    status staff_status NOT NULL DEFAULT 'Active'  
);
```

```
CREATE TABLE RENTAL (  
    rental_id SERIAL PRIMARY KEY,  
    rental_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    return_date TIMESTAMP,  
    customer_id INT REFERENCES CUSTOMER(customer_id) ON DELETE RESTRICT,  
    inventory_id INT REFERENCES INVENTORY(inventory_id) ON DELETE RESTRICT,  
    staff_id INT REFERENCES STAFF(staff_id) ON DELETE RESTRICT  
);
```

```
CREATE TABLE PAYMENT (  
    payment_id SERIAL PRIMARY KEY,  
    amount DECIMAL(7, 2) NOT NULL CHECK (amount > 0),  
    payment_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    status payment_status NOT NULL DEFAULT 'Completed',  
    rental_id INT REFERENCES RENTAL(rental_id) ON DELETE CASCADE,  
    customer_id INT REFERENCES CUSTOMER(customer_id) ON DELETE RESTRICT  
);
```

```
INSERT INTO CATEGORY (name) VALUES
```

('Action'), ('Comedy'), ('Sci-Fi'), ('Drama'), ('Horror');

INSERT INTO FILM (title, release_year, rental_rate) VALUES

('The Matrix', 1999, 4.99),

('Inception', 2010, 5.99),

('Interstellar', 2014, 5.99),

('The Hangover', 2009, 3.50),

('Pulp Fiction', 1994, 2.99);

INSERT INTO FILM_CATEGORY (film_id, category_id) VALUES

(1, 3), (1, 1),

(2, 3),

(3, 3), (3, 4),

(4, 2),

(5, 1), (5, 4);

INSERT INTO INVENTORY (film_id, condition) VALUES

(1, 'New'), (1, 'Good'),

(2, 'New'),

(3, 'Good'),

(5, 'Damaged');

INSERT INTO CUSTOMER (first_name, last_name, email) VALUES

('Oleksandr', 'Kovalenko', 'alex.koval@kpi.ua'),

('Mariia', 'Shevchenko', 'masha.sheva@gmail.com'),

('Dmytro', 'Moroz', 'dima_moroz@ukr.net'),

('Anna', 'Boyko', 'anna.boyko@outlook.com'),

('Artem', 'Tkachenko', 'artem.tkach@kpi.ua');

INSERT INTO STAFF (first_name, last_name, position, status) VALUES

('Ivan', 'Petrenko', 'Administrator', 'Active'),

('Olena', 'Lysenko', 'Cashier', 'Active'),

```
('Serhiy', 'Kravets', 'Cashier', 'Temporary'),
('Svitlana', 'Melnyk', 'Manager', 'On_Leave'),
('Andriy', 'Chorny', 'Cashier', 'Active');
```

```
INSERT INTO RENTAL (rental_date, return_date, customer_id, inventory_id, staff_id) VALUES
('2026-05-10 10:00:00', '2026-05-14 18:00:00', 1, 1, 2),
('2026-05-12 11:30:00', '2026-05-15 14:20:00', 2, 3, 2),
('2026-05-15 09:00:00', NULL, 3, 2, 3),
('2026-05-16 14:00:00', '2026-05-18 19:10:00', 4, 4, 5),
('2026-05-18 16:45:00', NULL, 1, 5, 5);
```

```
INSERT INTO PAYMENT (amount, payment_date, status, rental_id, customer_id) VALUES
(4.99, '2026-05-10 10:05:00', 'Completed', 1, 1),
(5.99, '2026-05-12 11:35:00', 'Completed', 2, 2),
(5.99, '2026-05-15 09:02:00', 'Completed', 3, 3),
(3.50, '2026-05-16 14:05:00', 'Completed', 4, 4),
(15.00, '2026-05-18 16:50:00', 'Delayed', 5, 1);
```

5. Результати виконання та докази працездатності

При виконанні вищезазначеного скрипту в Query Tool інтегроване середовище pgAdmin повернуло позитивний результат виконання транзакції без жодних повідомлень про синтаксичні помилки чи конфлікти обмежень цілісності ключів.

```
INSERT 0 5
Query returned successfully in 121 msec.
```

Для підтвердження успішного заповнення таблиць було виконано перевірочний запит до бази даних:

	customer_id [PK] integer	first_name character varying (100)	last_name character varying (100)	email character varying (150)
1	1	Oleksandr	Kovalenko	alex.koval@kpi.ua
2	2	Mariia	Shevchenko	masha.sheva@gmail.co...
3	3	Dmytro	Moroz	dima_moroz@ukr.net
4	4	Anna	Boyko	anna.boyko@outlook.c...
5	5	Artem	Tkachenko	artem.tkach@kpi.ua

База даних успішно зберігає інформацію, підтримує зв'язки та автоматично підставляє значення за замовчуванням (DEFAULT) для дат і початкових статусів.

Висновки

У ході виконання лабораторної роботи було успішно виконано проектування та фізичне розгортання реляційної схеми бази даних для пункту прокату фільмів у СКБД PostgreSQL.

Концептуальну ER-діаграму було декомпоновано на 8 взаємопов'язаних таблиць. Завдяки правильному порядку оголошення сутностей, налаштуванню кастомних типів ENUM, обмежень NOT NULL, UNIQUE та CHECK, було створено стійку архітектуру, яка захищає базу даних від некоректного введення інформації на системному рівні СУБД. Дані успішно завантажені та протестовані, що підтверджено результатами запитів у середовищі pgAdmin.